

算法

分数构成

平时 70 分：六次实验每次 10 分，考勤和课后作业 10 分。

期末 30 分：开卷考试。

课程内容与课本对应

1. 算法复杂度分析：第 1、2、3 章
2. 递归、分治策略：第 4 章
3. 堆、堆排序、二叉搜索树：第 6、12 章
4. 排序算法：第 7、8 章
5. 回溯法：课本无对应
6. 动态规划：第 15 章
7. 贪心算法：第 16 章 9
8. 摊还分析：第 17 章（这个应该不考）
9. 图算法：第 22-26 章
10. NP 理论：第 34 章

配着点书看这个吧，期末开卷也要熟悉书才好找知识点。

## 第 1 章 算法基本概念

这里应该是只考填空

### 基本概念

1. 算法：良定义的计算过程。必须满足 5 个性质：输入、输出、确定性、有限性、可行性。
2. 算法分析：预测算法所需的时间和空间资源。
3. 时间复杂度：算法在特定输入时执行的基本操作数或步数。它取决于两个因素：输入规模  $n$ 、具体的输入实例。

### 时间复杂度分析

1. RAM 模型假设：指令串行执行，每条基本指令耗时为常量，不考虑内存层次。
2. 三种运行情况：
  - 最好情况：给出下界。
  - 最坏情况：给出上界，**通常最关注**，因为它是任何输入的运行时间上限。
  - 平均情况：所有输入的期望，通常和最坏情况同阶。

### 渐近记号

把渐近记号看作  $n$  趋于无穷时函数增长率的比较，忽略低阶项和常数系数。

符号与算术对应：

- $O$  (大  $O$ )：渐近上界，相当于  $\leq$
- $\Omega$  (大  $\Omega$ )：渐近下界，相当于  $\geq$
- $\Theta$  (大  $\Theta$ )：渐近紧确界，相当于  $=$
- $o$  (小  $o$ )：非紧确上界，相当于  $<$
- $\omega$  (小  $\omega$ )：非紧确下界，相当于  $>$

性质：

- 传递性：所有符号都有。
- 自反性： $O$ 、 $\Omega$ 、 $\Theta$  有； $o$  和  $\omega$  没有。
- 对称性：只有  $\Theta$  有 ( $f = \Theta(g)$  等价于  $g = \Theta(f)$ )。
- 转置对称性： $O$  和  $\Omega$  互换， $o$  和  $\omega$  互换。

加法法则：算法由多部分组成时，整体复杂度由增长最快的那部分决定。

$$t_1(n) + t_2(n) \in O(\max(g_1(n), g_2(n)))。$$

复常见复杂度阶排序 (从小到大)：  $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$

## 第2章 递归与分治策略

这里可能要出 35+ 分,  $10 \times 2 + 15 \times 1 +$  有可能有填空

### 分治法基本概念

核心思想: 把大问题拆成相似的小问题, 递归解决后合并。

三个步骤:

1. 分解 (分): 拆成规模较小的子问题。
2. 求解 (治): 递归解子问题; 规模够小就直接解。
3. 组合 (合): 把子问题的解拼成原问题的解。

### 递归树法

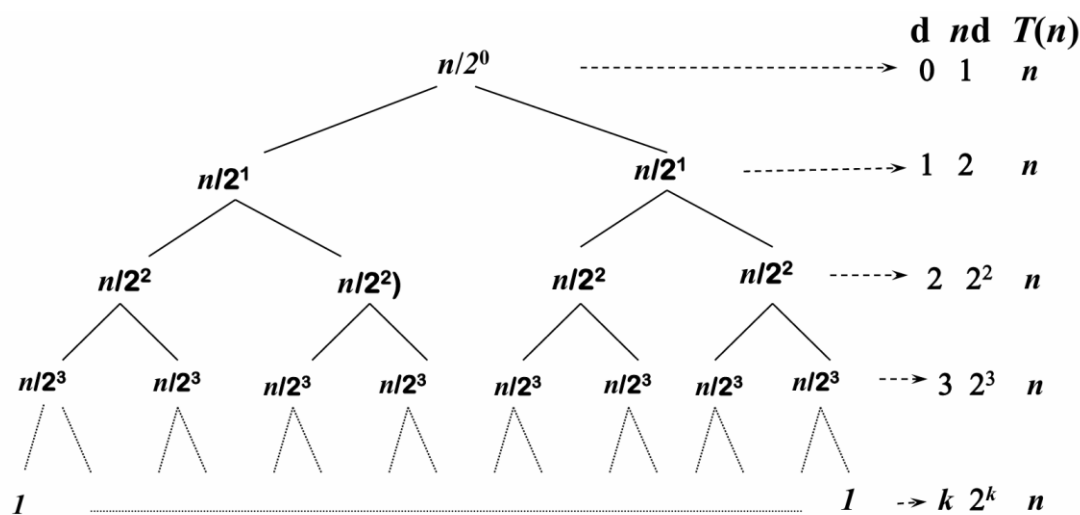
解题思路: 把递归式画成一棵树, 算出每层的代价总和, 再把所有层加起来。

画图步骤:

- 根节点是初始代价  $f(n)$ 。
- 每个节点分裂出  $a$  个子节点, 每个子节点规模变为  $n/b$ 。
- 标出每层的节点数、每个节点的代价、该层总代价。
- 算出树的深度 (通常是  $\log_b n$ )。
- 把所有层的代价求和。

e.g. 例 1: 归并排序用递归树求解时间复杂度

注意求和技巧



$$T(n) = (k + 1)(n) = (\lg n + 1)(n) = \Theta(n \lg n)$$

$$T(n) = 2T(n/2) + n$$

- 每层代价都是  $n$ , 共有  $\log_2 n + 1$  层。
- 总和:  $n(\log_2 n + 1) = \Theta(n \log n)$ 。

例 2:  $T(n) = 2T(n/2) + n^2$

-第  $i$  层有  $2^i$  个节点, 每个代价  $(n/2^i)^2$ , 层总代价为  $n^2/2^i$ 。

-这是一个等比数列求和, 首项  $n^2$ , 公比  $1/2$ 。

-总和:  $n^2(1 + 1/2 + 1/4 + \dots) = \Theta(n^2)$ 。

例 3:  $T(n) = 2T(n/2) + n/\log n$

-第  $i$  层总代价:  $2^i \times (n/2^i)/\log(n/2^i) = n/(\log n - i)$ 。

-总和:  $\sum_{i=0}^{\log n - 1} n/(\log n - i) = n \sum_{j=1}^{\log n} 1/j$ 。

-括号里是调和级数, 结果为  $\Theta(\log \log n)$ 。

-最终结果:  $\Theta(n \log \log n)$ 。

主定理法

专门解形式为  $T(n) = aT(n/b) + f(n)$  的递归式 ( $a \geq 1, b > 1$ )。

核心判断逻辑: 比较  $f(n)$  和 临界函数  $n^{\log_b a}$  谁增长得快。谁大选谁, 一样大就乘  $\log n$ 。

三种情况 (直接用即可):

-子问题主导:  $f(n)$  多项式意义上小于  $n^{\log_b a}$  (即  $f(n) = O(n^{\log_b a - \epsilon})$ )。

结论:  $T(n) = \Theta(n^{\log_b a})$ 。

-势均力敌: 两者同阶或相差对数因子 ( $f(n) = \Theta(n^{\log_b a} \log^k n)$ ,  $k \geq 0$ )。

结论:  $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$ 。

-合并代价主导:  $f(n)$  多项式意义上大于  $n^{\log_b a}$  (即  $f(n) = \Omega(n^{\log_b a + \epsilon})$ ), 且满足正则条件  $af(n/b) \leq cf(n)$  ( $c < 1$ )。

结论:  $T(n) = \Theta(f(n))$ 。

## 递归式求解方法2——主定理法 (2)

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } f(n) \in O(n^{\log_b a - \epsilon}), \epsilon > 0 & (1) \\ \Theta(n^{\log_b a} \lg^{k+1} n) & \text{if } f(n) \in \Theta(n^{\log_b a} \lg^k n), k \geq 0 & (2) \\ \Theta(f(n)) & \text{if } f(n) \in \Omega(n^{\log_b a + \epsilon}), \epsilon > 0 \text{ and } & (3) \end{cases}$$

$af(n/b) \leq cf(n)$  for some  
 $c < 1$  and all sufficiently large  $n$

正则条件

$$T(n) = 8T\left(\frac{n}{2}\right) + 1000n^2$$

$$T(n) = 2T\left(\frac{n}{2}\right) + 10n$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

例题：

例 1:  $T(n) = 5T(n/2) + n^2$

-临界函数:  $n^{\log_2 5}$  (约等于 $n^{2.32}$ )。

-比较:  $f(n) = n^2$  小于  $n^{2.32}$ , 属于第一种情况。

-答案:  $T(n) = \Theta(n^{\log_2 5})$ 。

例 2:  $T(n) = 27T(n/3) + n^3 \log n$

-临界函数:  $n^{\log_3 27} = n^3$ 。

-比较:  $f(n) = n^3 \log n$ , 相当于  $n^3 \log^1 n$ , 属于第二种情况 ( $k=1$ )。

-答案:  $T(n) = \Theta(n^3 \log^2 n)$ 。

例 3:  $T(n) = 5T(n/2) + n^3$

-临界函数:  $n^{\log_2 5}$ 。

-比较:  $f(n) = n^3$  大于  $n^{2.32}$ , 猜测属于第三种情况。

-验证正则条件:  $5(n/2)^3 = 5n^3/8 \leq cn^3$  (取  $c=5/8 < 1$ ), 成立。

-答案:  $T(n) = \Theta(n^3)$ 。

不能用情况举例:  $T(n) = 2T(n/2) + n/\log n$ , 因为  $n/\log n$  比  $n$  小, 但不是多项式意义上的小 (不满足第一种), 也不满足后两种。这题只能用递归树。

## 经典分治问题及复杂度分析

这部分主要应对填空, 简单记录几个提到的算法的思路。

### 1. 归并排序

-思路: 从中间劈开, 递归排左右, 合并两个有序数组。

-合并代价:  $O(n)$ 。

-递归式:  $T(n) = 2T(n/2) + O(n)$ , 结果  $\Theta(n \log n)$ 。

### 2. 最大子数组问题

-思路: 找中点, 递归求左半和右半的最大子数组。

关键在“合并”: 跨越中点的最大子数组, 从中点向左扫一遍求最大, 向右扫一遍求最大, 加起来。

-合并代价:  $O(n)$ 。

-递归式:  $T(n) = 2T(n/2) + O(n)$ , 结果  $\Theta(n \log n)$ 。

### 3. Strassen 矩阵乘法

-思路: 把矩阵分 4 块。常规分治需要 8 次乘法, Strassen 通过巧妙加减, 用 7 次乘法代替。

-递归式:  $M(n) = 7M(n/2)$  (这里是可以忽略加法代价的)。

-结果:  $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$ 。

### 4. 凸包问题

-思路：按  $x$  坐标排序，从中间劈开，递归求左右凸包。合并时，找左右凸包的上切线 and 下切线（通过旋转指针，每次方向检测  $O(1)$ ）。

-合并代价：  $O(n)$ 。

-递归式：  $T(n) = 2T(n/2) + O(n)$ ，结果  $\Theta(n \log n)$ 。

## 5.二维最近点对

-思路：按  $x$  坐标分两半，递归求左右最近距离  $d$ 。合并时，只需要检查距离分割线距离小于  $d$  的带状区域内的点（这里数学证明一侧最多 6 个点）。

-合并代价：  $O(n)$ 。

-递归式：  $T(n) = 2T(n/2) + O(n)$ ，结果  $\Theta(n \log n)$ 。

### 第3章 堆和堆排序

这里应该没大题，就简单过概念，小题翻书应该行

#### 基础概念

1. 二叉树性质：n 个结点的近似完全二叉树，高度为  $\Theta(\lg n)$ 。
2. 堆的数组实现：根是  $A[1]$ ，节点  $i$  的左孩子  $2i$ ，右孩子  $2i+1$ ，父节点  $\lfloor i/2 \rfloor$ 。
3. 堆性质：最大堆（父  $\geq$  子，根最大），最小堆（父  $\leq$  子，根最小）。

#### 核心操作与复杂度

##### 1. Max-Heapify（维护堆性质）

- 前提： $i$  的左右子树已经是最大堆。
- 过程：把  $i$  和较大的孩子交换，一直向下“沉”。
- 复杂度： $O(\lg n)$ ，运行时间与节点高度成正比。

##### 2. Build-Max-Heap（建堆）

- 过程：从最后一个非叶节点  $\lfloor n/2 \rfloor$  开始，倒序遍历到根，依次调用 Max-Heapify。
- 复杂度：简单界是  $O(n \lg n)$ ，**准确界是  $O(n)$** 。  
*关于为什么是  $O(n)$ ：因为堆里大多数节点高度很低，按高度求和是一个收敛的等比数列，最后结果被  $n$  限制。*

##### 3. Heap Sort（堆排序）

- 过程：先建最大堆，然后把根（最大值）和数组末尾交换，堆大小减 1，对新根调用 Max-Heapify。重复直到剩 1 个元素。
- 复杂度： $O(n \lg n)$ ，且是原地排序。

#### 优先队列

##### 1. 基础操作（复杂度都是 $O(\lg n)$ ）：

- Heap-Extract-Max：根和末尾交换，堆大小减 1，对新根 Max-Heapify。
- Heap-Increase-Key：把节点值改大，然后不断和父节点比较交换（向上浮）。
- Max-Heap-Insert：在末尾加个  $-\infty$ ，然后调用 Increase-Key。

##### 2. 改进 Find 操作

- 问题：普通堆找特定 id 的元素需要遍历， $O(n)$ 。
- 改进：引入辅助数组  $L$  (handle)， $L[i]$  直接存 id 为  $i$  的元素在堆数组  $A$  中下标。
- 效果：Find 时间降为  $O(1)$ 。代价是堆里元素位置变动时，必须同步更新  $L$ 。

## 第 4 章 回溯法与博弈树 猜测大题/10 分，注意这章书上好像没有，要记忆

### 核心定义与解空间（概念）

解向量：问题的解通常表示为一个  $n$  元组  $X = (x_1, x_2, \dots, x_n)$ 。

例如  $N$  皇后问题， $x_i$  表示第  $i$  行皇后所在的列号。

解空间：所有可能解的集合。通常组织成树状结构（状态空间树）。

子集树：当问题是从  $n$  个元素中选子集时（如 0-1 背包、子集和）。树高  $n$ ，叶子节点数  $2^n$ 。

排列树：当问题是确定  $n$  个元素的排列时（如 TSP 旅行商、 $N$  皇后）。树高  $n$ ，叶子节点数  $n!$ 。

三类解：

可能解：解空间中的任意节点。

可行解：满足约束条件的解。

最优解：使目标函数取极值（最大/最小）的可行解。

复杂度分析：

时间复杂度：最坏情况下需要遍历整棵树， $O(2^n)$ （子集树）或  $O(n!)$ （排列树）。

空间复杂度：回溯法使用 DFS，任何时刻只保存从根到当前节点的路径，空间复杂度为  $O(n)$ （即树的高度）。

### 回溯法框架与剪枝策略

回溯法=DFS+剪枝函数。

通用框架：

```
void backtrack(int t) {
    if (t > n) output(x); // 到达叶子，输出解
    else {
        for (int i = f(n,t); i <= g(n,t); i++) {
            x[t] = h(i); // 尝试赋值
            if (constraint(t) && bound(t)) // 剪枝判断
                backtrack(t + 1);
        }
    }
}
```

两种剪枝函数：

约束函数：剪去不满足约束的子树（即不可行解）。

例： $N$  皇后中，如果当前列已被占用，剪枝。

例：0-1 背包中，如果当前重量 > 容量，剪枝。



限界函数：剪去不可能产生更优解的子树（用于求最优解）。

例：当前节点的上界价值  $\leq$  目前已找到的最优价值，剪枝。

## 界限函数

通常出现在 0-1 背包或装载等问题中。

什么是界限函数  $B(X)$ ？

定义：对于状态空间树中的节点  $X$ ， $B(X)$  表示以  $X$  为根的子树中，可能达到的目标函数值的上界（对于最大化问题）。

计算方法：通常使用贪心策略（如分数背包问题）来快速估算上界。

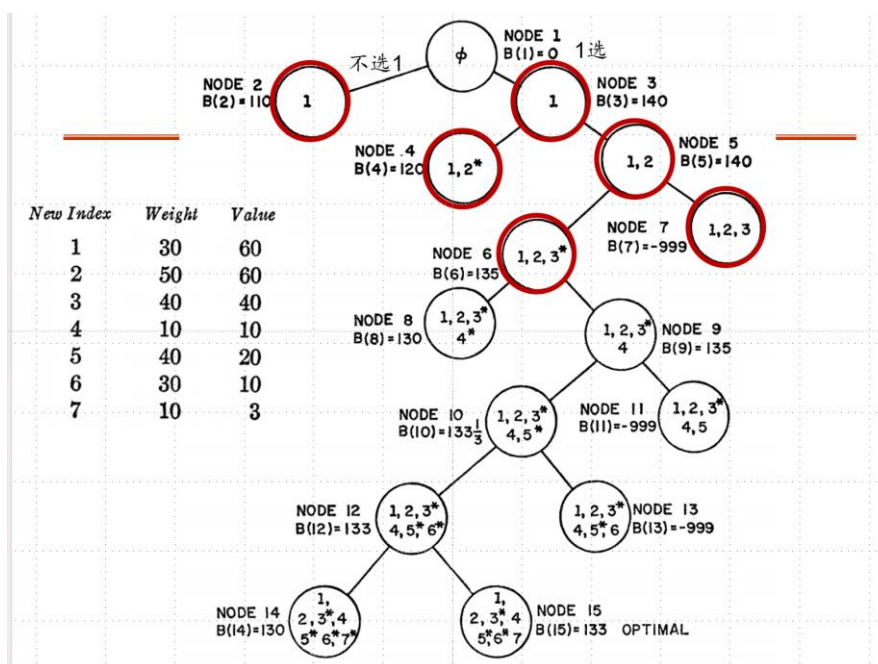
$B(X)$  = 当前已选价值 + 剩余容量能装下的最大分数价值。

剪枝逻辑：

维护一个全局变量  $bestValue$ （当前找到的最优解）。

如果  $B(X) \leq bestValue$ ，说明这条路即使走到黑也超不过现在的最好结果，剪枝。

举个例子-分数背包问题求解， $w=100$  情况



这棵树是按深度逐层生长，每次走“不超限制的  $B(i)$  最大侧”得到

## 经典回溯问题及思路

### N 皇后问题

解向量： $X = (x_1, \dots, x_n)$ ， $x_i$  是第  $i$  行皇后的列号。

约束条件：

不同列： $x_i \neq x_j$

不同斜线： $|i - j| \neq |x_i - x_j|$ （行差绝对值  $\neq$  列差绝对值）

思路：逐行放置，每放一个检查是否与上面冲突。

### 子集和问题

问题：从集合  $W = \{w_1, \dots, w_n\}$  中找子集，和为  $S$ 。

剪枝条件：设当前和为  $\text{weightSoFar}$ ，剩余所有元素和为  $\text{totalPossibleLeft}$ 。节点“有希望”需同时满足：

$\text{weightSoFar} + \text{totalPossibleLeft} \geq S$ （剩下的全加上也达不到  $S$ ）

$\text{weightSoFar} + w_{i+1} \leq S$ （加上最小的下一个就超了）

### 0-1 背包/装载问题

思路：子集树。左分支选 ( $x_i = 1$ )，右分支不选 ( $x_i = 0$ )。剪枝：

右子树（选）：检查是否超重。  $cw + w[i] \leq C$ 。

左子树（不选）：检查是否有希望。  $cw + r > \text{bestw}$  ( $r$  是剩余物品总重)。

如果即使剩下全装上也不超过当前最优解，剪枝。

### 地图填色（图着色）

优化策略（MRV 准则）：优先给可选颜色最少（约束最多）的区域涂色。

思想：最容易失败的先试，早点发现死胡同。

### 博弈树：Minimax 与 Alpha-Beta 剪枝

#### Minimax 算法

场景：双人零和博弈（如井字棋、象棋）。

角色：Max 层（我方）：选择子节点中估值最大的。

Min 层（对手）：选择子节点中估值最小的（假设对手极其聪明，想让我输）。

评估函数：对于非叶子节点用函数估算局势优劣（如棋子价值差）。

#### Alpha-Beta 剪枝

在 Minimax 基础上，通过剪枝减少搜索量，不影响最终结果。

两个参数：

$\alpha$ ：Max 节点目前能保证的最低收益（下界）。初始  $-\infty$ 。

$\beta$ ：Min 节点目前能保证的最高损失（上界）。初始  $+\infty$ 。

剪枝规则： $\alpha$  剪枝：发生在 Min 节点。

如果 Min 节点发现某个子节点的值  $v \leq \alpha$ ，说明 Max 节点（父节点的父节点）绝不会选这条路（因为 Max 已经有更好的  $\alpha$  了）。

动作：剪掉 Min 节点剩余的子节点。

$\beta$  剪枝：发生在 Max 节点。

如果 Max 节点发现某个子节点的值  $v \geq \beta$ ，说明 Min 节点（父节点）绝不会让这条路发生（因为 Min 已经有更小的  $\beta$  了）。

动作：剪掉 Max 节点剩余的子节点。

## 第 5 章 贪心法      猜测大题/10 分，填空也可能涉及

核心概念：

贪心法是一种通用算法策略。

核心思想：基于贪心选择准则，每次做出局部最优的选择，希望通过局部最优达到全局最优。

与动态规划(DP)的区别：贪心法不回溯、不修改以前的选择，内存更高效、速度更快，但缺点是不能保证对所有问题都得到最优解。

贪心准则与经典问题：贪心法的关键在于“选择什么准则”，不同准则结果完全不同。

找零问题：准则：优先选面额最大的硬币。

注意：对常规面额(如 25,10,5,1)能得到最优解；但对任意面额不一定最优(如面额 25,10,1 凑 30，贪心需要 6 枚，最优是 3 枚 10)。

背包问题：

0/1 背包：无论用最大价值、最小体积还是最大单位价值优先，都不能保证最优。

分数背包：**最大单位价值( $p/w$ )**优先。能保证最优解。

活动选择问题：给定  $n$  个活动的开始和结束时间，找最大兼容活动子集。

准则：**最早结束时间**优先。

思路：按结束时间升序排序，依次选择与已选活动不冲突且结束最早的活动。

任务调度：准则：优先运行时间最长的任务。**不一定是最优解**，但解通常不坏。

什么情况能得到最优解？（怎么证明）

要证明贪心法对某个问题有效，必须证明该问题具备两个性质：

贪心选择性质

局部最优能导致全局最优。即第一步的贪心选择一定包含在某个最优解中。

证明思路-替换法：假设最优解  $A$  不包含贪心选择  $a_1$ 。找到  $A$  中第一个结束的活动  $a$ ，用  $a_1$  替换  $a$ 。因为  $a_1$  结束时间更早，替换后不会与后面的活动冲突，且活动总数不变，所以新解  $A^*$  也是最优解。这就证明了贪心选择可行。

最优子结构

做出贪心选择后，剩下的子问题的最优解与贪心选择组合是原问题的最优解。

证明思路：假设原问题最优解是  $A$ ，包含贪心选择  $a_1$ 。那么  $A$  去掉  $a_1$  后剩下的部分，必然是剩余子问题的最优解。如果不是，就能找到更好的子问题解，与  $A$  是最优解矛盾。

设计思路：

将原问题分解为若干个子问题。确定合适的贪心选择准则（感觉通常需要排序？）。

证明贪心选择性质和最优子结构。根据准则，自顶向下，以迭代的方式做出贪心选择，将问题规模缩小，直到问题求解完毕。

## 第 6 章 动态规划

应该大题\*1, 猜测 15 分, 可能涉及伪代码, 另外填空同样可能

### 基本概念与术语

#### 核心定义

- 动态规划: 将多阶段决策问题转化为一系列单阶段问题求解
- 多阶段决策问题: 问题可划分为相互联系的阶段, 每阶段需做决策, 上阶段决策影响下阶段

#### 关键术语

- 阶段: 把问题求解过程划分为若干相互联系的阶段
- 状态: 每个阶段开始时问题的客观状况
- 状态的无后效性: 某阶段状态给定后, 以后过程的发展不受该阶段以前各阶段状态的影响
- 策略: 各个阶段决策组成的决策序列
- 子策略: 由某个阶段开始到终止阶段的决策序列

#### 适用条件 (DP 有效的两个必要条件)

- 最优子结构: 问题的最优解由其子问题的最优解构造
- 重叠子问题: 递归求解时, 有些子问题被反复计算多次

### 动态规划基本思想

#### 与分治法的比较

- 相同点: 都将原问题分解为子问题, 先求解子问题, 再从子问题解得到原问题解
- 不同点:
  - 分治法: 子问题相互独立, 共同部分被重复计算
  - 动态规划: 子问题不独立, 保存已解决的子问题答案, 避免重复计算

#### 两种实现方式

- 带备忘的自顶向下法: 保存每个子问题的解, 需要时先检查是否已保存, 是则直接返回, 否则计算并保存
- 自底向上法: 将子问题按规模排序, 由小到大求解, 求解某子问题时, 它所依赖的更小子问题都已求解完毕, 每个子问题只求解一次

### 动态规划设计步骤

- 找出最优解的性质, 刻画其结构特征
- 递归地定义最优值 (写出 DP 方程)
- 以自底向上的方式计算出最优值
- 根据计算最优值时记录的信息, 构造最优解

### 经典问题列举与举例详解

#### 钢条切割问题

**问题描述:** 给定长度为  $n$  的钢条和价格表  $p_i (i=1,2,\dots,n)$ , 求切割方案使收益  $r_n$  最大

**最优子结构:** 定义  $r_k$  为长度为  $k$  的钢条的最优切割代价

递推关系:  $r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$

DP 方程:  $r_n = \begin{cases} 0 & \text{if } n = 0 \\ \max_{1 \leq i \leq n} (p_i + r_{n-i}) & \text{if } n > 0 \end{cases}$

时间复杂度: 递归实现:  $O(2^n)$ , 自底向上 DP:  $O(n^2)$ , 带备忘自顶向下:  $O(n^2)$

其他常见的还有: 背包问题, 矩阵链乘法, 最长公共子序列 (LCS), 切杆问题变种: 每次切割代价为  $c$ , 用 DP 实现斐波那契数  $F(n) = F(n-1) + F(n-2)$ , 带记忆的递归 LCS 算法, Coins in a Line 游戏, 投骰子问题, 能量项链问题

## DP 设计技巧与问题

### 设计技巧

- 阶段划分: 直接影响计算复杂性
- 状态表示: 影响存储表设计
- 记忆型递归: DP 的变种, 第一次遇到子问题计算并填表, 以后查表

### 存在的问题

- 阶段划分和状态表示需具体问题具体分析, 无通用方法
- 空间溢出: 需要很大空间存储中间结果
- 数据不易压缩存储

## 第 7 章 图论算法

这里许多之前学过，笔记只记录新知识/个人认为重点部分：

### 并查集

#### 基本概念

作用：维护不相交集合，支持快速查找和合并操作

应用：Kruskal 算法中判断两个顶点是否属于同一连通分支

#### 基本操作

**Make-Set(x)**: 创建包含元素  $x$  的新集合,  $O(1)$

**Find-Set(x)**: 返回包含  $x$  的集合的代表元素,  $O(1)$  (使用路径压缩)

**Union(x, y)**: 合并包含  $x$  和  $y$  的两个集合,  $O(\alpha(n)) \approx O(1)$

#### 实现方式

用链表或树结构实现

每个集合有一个代表性元素 (通常选 ID 最小的)

#### 优化:

按秩合并/路径压缩

### 流网络

#### 基本概念

##### 流网络定义:

有向图  $G=(V, E)$ , 每条边  $(u,v)$  有非负容量  $c(u,v) \geq 0$

两个特殊顶点: 源点  $s$  和汇点  $t$

每个顶点都在从  $s$  到  $t$  的某条路径上

##### 流的三个性质:

容量限制:  $f(u,v) \leq c(u,v)$

反对称性:  $f(u,v) = -f(v,u)$

流守恒性: 对所有  $u \in V - \{s, t\}$ ,  $\sum f(u,v) = 0$

流的值:  $|f| = f(s, V) =$  从源点流出的总流量

#### 最大流问题

问题: 找到从  $s$  到  $t$  的最大流量

#### Ford-Fulkerson 方法

##### 核心思想:

迭代方法, 初始流为 0

每次找到一条**增广路径** (从  $s$  到  $t$  的路径, 每条边还有剩余容量)

沿增广路径增加流量

重复直到没有增广路径

##### 关键概念:

##### 残留网络 $G_f$ :

残留容量:  $cf(u,v) = c(u,v) - f(u,v)$

包含正向边 (还有剩余容量) 和反向边 (可以退还流)

$$|Ef| \leq 2|E|$$

**增广路径 p:**

残留网络中从 s 到 t 的简单路径

残留容量:  $cf(p) = \min\{cf(u,v) \mid (u,v) \text{ 在 } p \text{ 上}\}$

**算法伪代码:**

```
Ford-Fulkerson(G, s, t)
  for each (u,v) ∈ E(G)
    do f[u,v] ← 0
       f[v,u] ← 0
  while there exists a path p from s to t in Gf
    do cf(p) ← min{cf(u,v) : (u,v) is in p}
       for each (u,v) in p
         do f[u,v] ← f[u,v] + cf(p)
            f[v,u] ← -f[u,v]
  return f
```

## 最大流最小割定理

**割(S, T):**

将 V 划分为 S 和  $T=V-S$ , 其中  $s \in S, t \in T$

割的容量:  $c(S,T) = \sum c(u,v), u \in S, v \in T$

割的净流:  $f(S,T) = |f|$  (任意割的净流都相等)

**定理:** 以下三个条件等价:

f 是 G 的一个最大流

残留网络  $G_f$  不包含增广路径

对 G 的某个割(S,T), 有  $|f| = c(S,T)$

**推论:** 最大流的值 = 最小割的容量

## Edmonds-Karp 算法

**改进:**

用 BFS 实现 Ford-Fulkerson

每次选择**最短增广路径** (边数最少)

**时间复杂度:**  $O(VE^2)$

增广次数:  $O(VE)$

每次 BFS:  $O(E)$

**性质:** 残留网络中最短路径长度  $\delta_f(s,v)$  随每次流增加而单调递增

## 其他图算法 (已学的仅列名, 没学的简析)

1. 图的遍历: **BFS/DFS**

2. 拓扑排序: 应用于 DAG 的线性排序

3. 强连通分支

#### 4. 最小生成树：Kruskal 算法/Prim 算法

#### 5. 最短路径：Dijkstra 算法

这里还有一个 Bellman-Ford 算法：可处理负权边， $O(VE)$ ，简析：

是单源最短路径算法。

优势：能处理带负权边的图，并且能检测负权环。

缺点：比 Dijkstra 慢。

核心思路（松弛操作）

初始化：源点距离为 0，其他所有点距离为无穷大。

松弛：对图中的**每一条边** $(u, v)$ ，检查  $\text{dist}[u] + w(u, v) < \text{dist}[v]$  是否成立，如果成立就更新  $\text{dist}[v]$ 。

循环：将上述“对所有边松弛”的操作，重复执行  **$V-1$  次**（ $V$  是顶点数）。因为最短路径最多包含  $V-1$  条边。

负环检测：再额外执行一次松弛操作，如果还能更新距离，说明图中存在负权环。



## 第 9 章 NP 问题

### 核心概念辨析

P 问题：能在多项式时间内**求解**的问题。

NP 问题：能在多项式时间内**验证**一个解是否正确的问题。

NPC 问题：NP 问题中最难的一类。所有 NP 问题都能在多项式时间内**归约**到它。如果任何一个 NPC 问题找到了多项式求解算法，则  $P=NP$ 。

NP-Hard 问题：至少和 NP 问题一样难。所有 NP 问题都能归约到它，但它**不一定是 NP 问题**（可能连验证解都无法在多项式时间内完成）。

### 包含关系与归约

包含关系（从小到大）： $P \subseteq NP \subseteq NPC \subseteq NP\text{-Hard}$

归约：问题 A 可以归约为问题 B，意味着用 B 的解法能解决 A，说明 A 不比 B 难。

证明一个问题是 NPC 的思路：找到一个已知的 NPC 问题，把它归约到这个问题上

### 经典问题归类

属于 NPC（既是 NP 又是 NP-Hard）：

SAT（布尔可满足性问题，第一个被证明的 NPC），TSP（旅行商问题），哈密尔顿回路问题，图染色问题，顶点覆盖问题，0/1 背包问题（这里注意分数背包是 P 问题，贪心可解）

属于 P 问题（多项式可解）：排序、最短路径、最小生成树、分数背包。

### 停机问题

定义：给定一个程序和输入，判定这个程序是否会停机。

结论：图灵证明了停机问题是**不可解的**（不存在通用算法能判定）。

定性：停机问题是 **NP-Hard**，但**不是 NPC**，原因：它连“在多项式时间内验证解”都做不到，所以它根本不是 NP 问题，自然也不是 NPC。